4.2 Logistic regression

Vladimir Buskin

English Language and Linguistics, Catholic University of Eichstätt-Ingolstadt

ORCID: 0009-0005-5824-1012

Last updated: 14 October 2026, 21:47

This handout introduces logistic regression for modelling binary response variables, covering log odds and odds ratios, maximum likelihood estimation, model fitting and diagnostics in R, and the interpretation and visualisation of model output, using corpus data on subject pronoun realisation in varieties of English.

Table of contents

Recommended reading . . . 2
Preparation . . . 2
Descriptive overview . . . 3
Logistic regression . . . 5
The simple logistic model . . . 6
Multiple logistic regression . . . 7
Odds ratios . . . 8
Maximum likelihood estimation . . . 8
Workflow in R . . . 10
Research question and hypotheses . . . 10
Convert to factors and specify reference levels . . . 10
Model fitting . . . 11
Further model diagnostics . . . 13
Confidence intervals and odds ratios . . . 15
Visualisation . . . 16
Interpret the model . . . 18
Looking under the hood . . . 18
Some hidden metrics . . . 18
How do you actually estimate β in logistic regression? . . . 21
References . . . 22

Recommended reading

For linguists:

> Levshina (2015: Chapter 12)
>
> Winter (2020: Chapter 12)

Full theoretical treatment:

> James et al. (2021: Chapter 4)
>
> Hosmer & Lemeshow (2008)

Preparation

> **Script**
>
> You can find the full R script associated with this unit here.

Consider the data from Buskin's (2025)[1] corpus study on subject pronoun realisation:

```
# Load libraries
library(tidyverse)
library(broom)
library(DescTools) # Pseudo-R2
library(lmtest) # likelihood ratio test
library(car) # variance inflation factors

# Load data
data_pro <- read.csv("INPUT_pronouns.csv", sep = ",", header = TRUE)

# Inspect data
str(data_pro)
head(data_pro)
```

- **Target variable**:
  - Type of pronominal `Reference` ('overt', 'null')
- **Explanatory variables**:
  - Grammatical `Person` ('1', '2', '3' as well as the dummy pronouns 'it' and 'there')

[1]The input data can be downloaded from this OSF repository: https://osf.io/qgnms.

Variety	Reference	n	pct
GB	null	29	1.817043
GB	overt	1567	98.182957
HK	null	75	4.507212
HK	overt	1589	95.492789
SING	null	70	4.435995
SING	overt	1508	95.564005

- `Register` (the text category in the International Corpus of English; 'S1A' comprises informal private conversations, whereas 'S1B' comprises public dialogues such as class lessons)
- `Variety` of English (British English 'GB', Singapore English 'SING' and Hong Kong English 'HK'), and
- `Referentiality` ('referential' with an identifiable referent or 'non-referential' with no/generic reference)

```
head(data_pro)
```

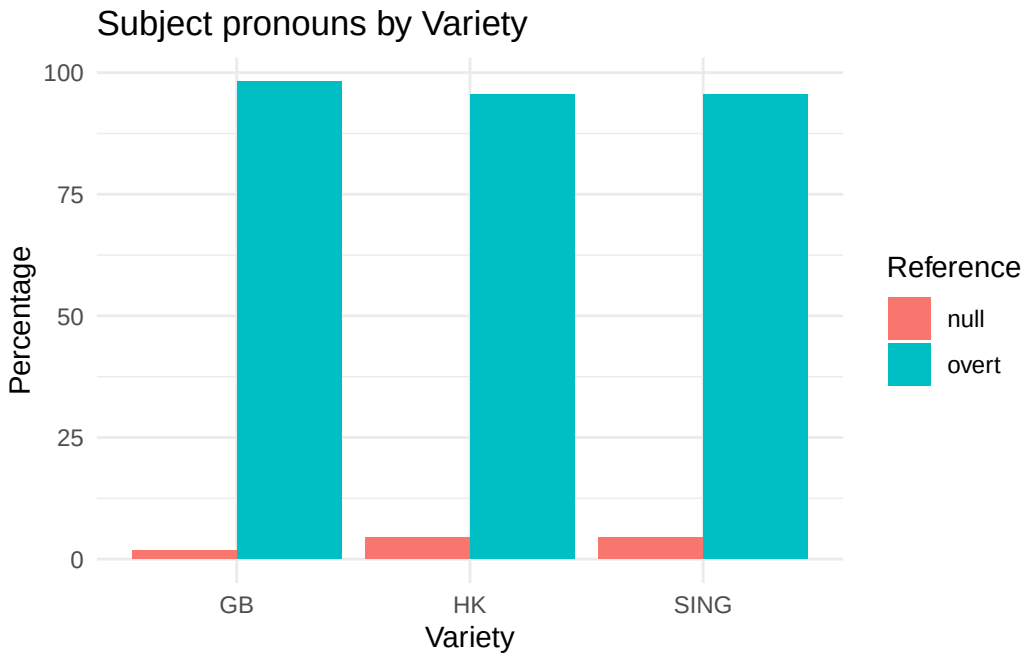
```
  Reference Person Register Variety Referentiality
1     overt      3      S1A      GB    referential
2     overt      3      S1A      GB    referential
3     overt      3      S1A      GB    referential
4     overt      3      S1A      GB    referential
5     overt      3      S1A      GB    referential
6     overt      3      S1A      GB    referential
```

```
table(data_pro$Reference)
```

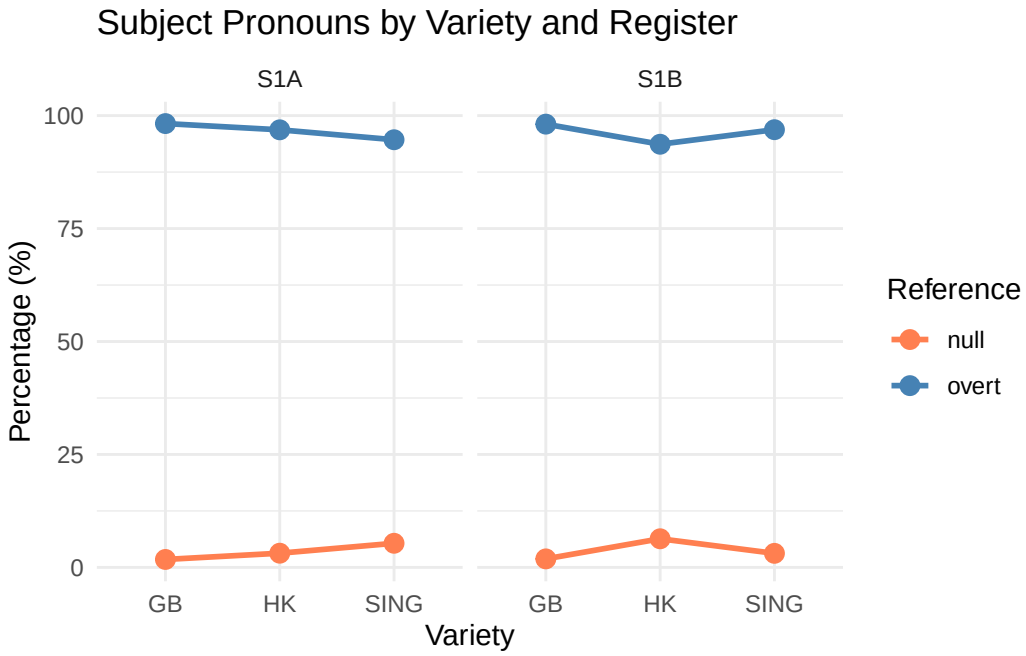
```
 null overt
  174  4664
```

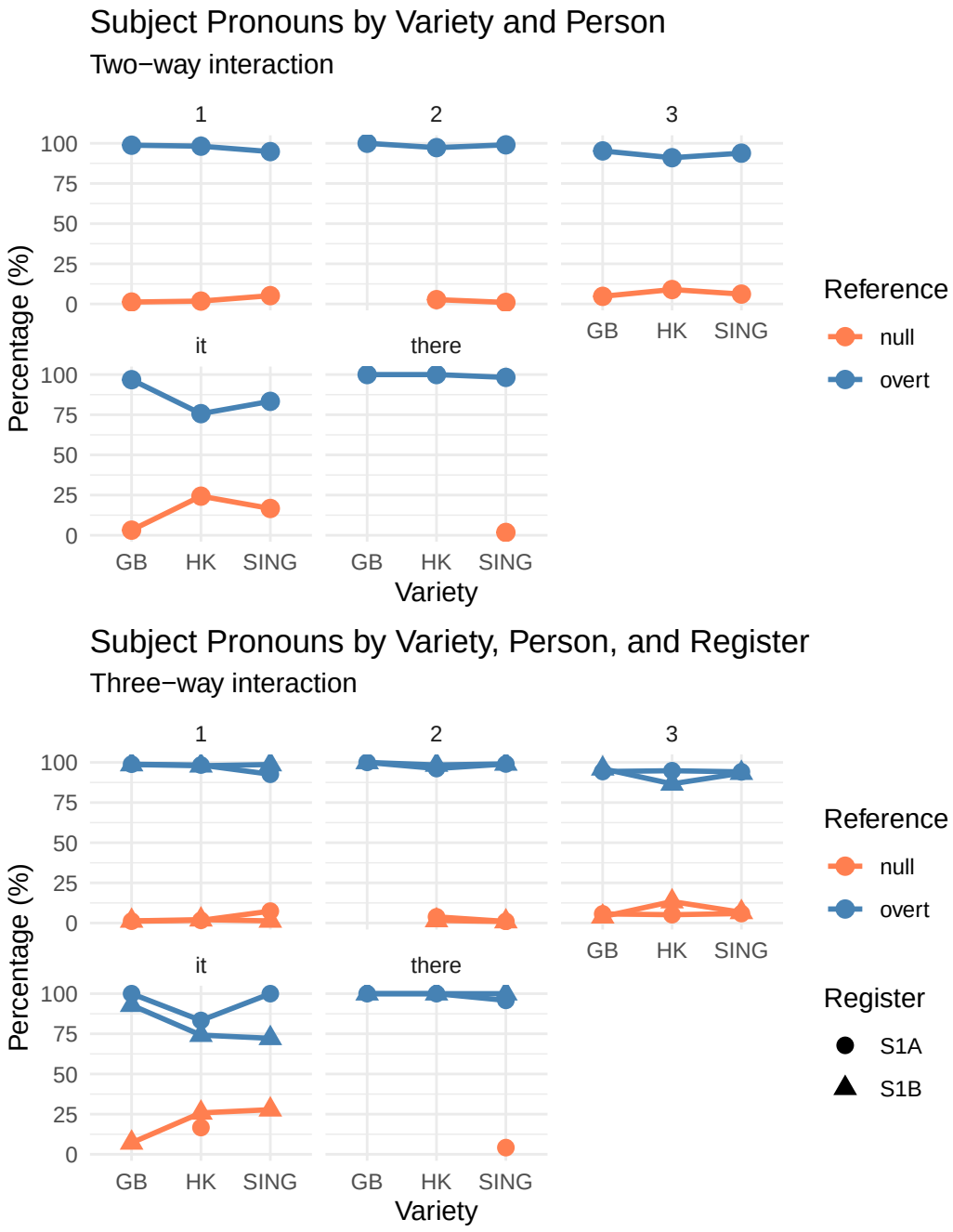
Descriptive overview

```
data_pro_stats1 <- data_pro %>%
  group_by(Variety) %>%
  count(Reference) %>%
  mutate(pct = n/sum(n) * 100)
```




```
data_pro_stats2 <- data_pro %>%
  group_by(Variety, Register) %>%
  count(Reference) %>%
  mutate(pct = n/sum(n) * 100)
```





Logistic regression

In contrast to linear regression, logistic regression models a **discrete response variable** Y with two outcomes[2]. In the present study, Y is pronominal `Reference` and has the out-

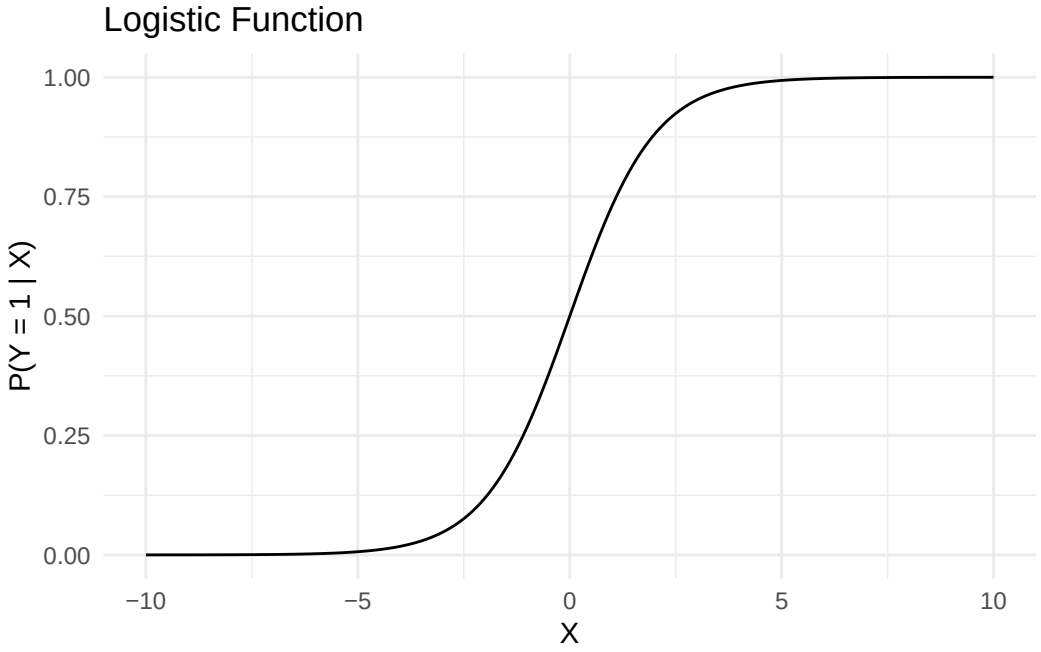
[2]Logistic regression can be generalised to response variables with ≥ 3 unordered classes, which is known as **multinomial logistic regression** or **softmax regression**. For ordered classes, see 4.5 Ordinal regression.

comes `Reference = null` and `Reference = overt`, which represent null and overt subjects, respectively. Dichotomous variables of this kind are also often coded as `yes/no` or `1/0`.

Another difference from linear regression is the **output** of the model:

- In linear regression, we obtain a predicted value for the continuous response variable we're interested in. For instance, if we're modelling reaction times, the model will return an estimated mean reaction time (given the predictors).
- In logistic regression, the model will return a **probability** (namely that of $Y = 1$, given the predictors). In variational linguistics, this may correspond to the probability that a speaker will use one syntactic variant versus the other.

A core component of logistic regression is the **logistic function**. The rationale for using it is that the output of the function will always lie between 0 and 1 and can thus be interpreted as a probability.



The simple logistic model

Assuming a binary response variable Y with the values 1 and 0 and a single predictor X, the conditional probability $P(Y = 1 \mid X)$ is then modelled via the logistic function (also known as the inverse logit) in Equation 1.

$$P(Y = 1 \mid X) = \frac{e^{\beta_0 + \beta_1 X}}{1 + e^{\beta_0 + \beta_1 X}}. \tag{1}$$

With some algebraic manipulation it can be shaped into a form that is definitely more familiar:

$$\log\left(\frac{P(Y=1 \mid X)}{1-P(Y=1 \mid X)}\right) = \beta_0 + \beta_1 X. \tag{2}$$

The logistic model has several characteristic components. The fraction $\frac{P(Y=1|X)}{1-P(Y=1|X)}$ represents the **odds**, i.e., the probability of one outcome (e.g., `Reference = null`) relative to the probability of the other (e.g., `Reference = overt`). Their logarithm is known as the **log odds** (or **logit**) of one outcome versus the other.

> Understanding log odds
>
> When interpreting the output of a logistic model, note that
>
> - log odds above 0 correspond to probabilities above 0.5 (the outcome $Y = 1$ is more likely than not), whereas log odds below 0 correspond to probabilities below 0.5;
> - positive coefficients indicate that the odds $\frac{P(Y=1|X)}{1-P(Y=1|X)}$ **increase** as X increases (or relative to the reference level of a categorical predictor), whereas
> - negative coefficients indicate that these odds **decrease**.

In more concrete terms: If we are interested in the probability that the form of pronominal reference is `null` (our Y) while taking into account the extra-linguistic context (`Register`; our X), the model would then have the general form in Equation 3.

$$\log\left(\frac{P(\text{Reference} = \text{null} \mid \text{Register})}{1-P(\text{Reference} = \text{null} \mid \text{Register})}\right) = \beta_0 + \beta_1 \text{Register} \tag{3}$$

Multiple logistic regression

If more than one predictor is included, the above equations can be expanded so as to take into account p slopes for p independent variables $X_1, X_2, ..., X_p$.

$$P(Y=1 \mid X_1, ..., X_p) = \frac{e^{\beta_0+\beta_1 X_1+...+\beta_p X_p}}{1+e^{\beta_0+\beta_1 X_1+...+\beta_p X_p}}. \tag{4}$$

Consequently, the log odds correspond to the linear predictor $\beta_0 + \beta_1 X_1 + \beta_2 X_2 + \cdots + \beta_p X_p$ (cf. Equation 5).

$$\log\left(\frac{P(Y=1 \mid X_1, ..., X_p)}{1-P(Y=1 \mid X_1, ..., X_p)}\right) = \beta_0 + \sum_{i=1}^{p} \beta_i X_i \tag{5}$$

Odds ratios

To assess the strength of an effect, it is instructive to examine the **odds ratios** that correspond to the model coefficients. Odds ratios (OR) are defined as

$$OR(X_1) = e^{\beta_1}.$$

The OR indicates the factor by which the odds of $Y = 1$ change if X_1 increases by one unit or, for a categorical predictor, if we move from the reference level to another level. An OR of 1 corresponds to no effect.

> Understanding odds ratios
>
> Essentially, the OR describes the ratio between two odds, i.e., how the odds of an outcome differ between two levels of an independent variable. This is illustrated for `Reference` given `Register` below (with S1A as the reference level):
>
> $$\text{OR}(\text{Reference} \mid \text{Register}) = \frac{\frac{P(\text{Reference=null}|\text{Register=S1B})}{P(\text{Reference=overt}|\text{Register=S1B})}}{\frac{P(\text{Reference=null}|\text{Register=S1A})}{P(\text{Reference=overt}|\text{Register=S1A})}}$$
>
> Read as: '**The odds** of a null (vs. overt) subject in S1B **divided by** the odds of a null (vs. overt) subject in S1A'.

Maximum likelihood estimation

In contrast to linear regression, the parameters of a logistic regression model cannot be computed with a closed-form formula. Rather than finding a regression line that minimises the squared distances to all data points, the default approach of logistic models is to find the parameter values under which the observed data are **most likely**. Hence this procedure is known as **maximum likelihood estimation** (MLE).

The model first makes an assumption about the probability distribution of the data. For categorical data, the binomial distribution is a common choice. Equation 6 indicates the corresponding probability mass function, which describes the probability of observing y successes in k independent trials, each with a success probability of π.

$$f(y; k, \pi) = \binom{k}{y} \pi^y (1-\pi)^{k-y} \tag{6}$$

Maximum likelihood estimation turns this question around: If we tossed a coin $k = 10$ times and observed $y = 5$ heads (i.e., 5 successes), which value of π would make this outcome most likely? (Unsurprisingly, the answer is $\hat{\pi} = 0.5$.)

Now, let β denote some parameter of interest (e.g., the slope coefficient of a logistic regression model). Given some observed data, the likelihood of this parameter can be described in

terms of the likelihood function $L(\beta)$ in Equation 7. It treats each of the n observations as a single trial ($k = 1$), for which the binomial coefficient equals 1, and computes the product of their probabilities. Here, π_i denotes the probability of $Y = 1$ for observation i, which depends on β via Equation 4. For each data point, only one of the two factors is relevant: π_i if $y_i = 1$ and $1 - \pi_i$ if $y_i = 0$.

$$L(\beta) = \prod_{i=1}^{n} \pi_i^{y_i} (1 - \pi_i)^{1-y_i} \tag{7}$$

Within the context of our pronoun data (with $\text{null}_i = 1$ for null subjects and 0 for overt ones), this can be understood more intuitively as the product, across all subjects, of the predicted probabilities of the variants that were actually observed:

$$L(\beta) = \prod_{i=1}^{n} \pi_i^{\text{null}_i} (1 - \pi_i)^{1-\text{null}_i} \tag{8}$$

Conventionally, this expression is log-transformed in order to convert the product into a sum, which is easier to handle computationally (multiplying thousands of small probabilities would quickly result in numbers too small for a computer to represent). The log-likelihood function $\ell(\beta)$ in Equation 9 forms the basis for a variety of goodness-of-fit measures used to evaluate logistic regression models.

$$\ell(\beta) = \sum_{i=1}^{n} [y_i \log(\pi_i) + (1 - y_i) \log(1 - \pi_i)] \tag{9}$$

And in more concrete terms:

$$\ell(\beta) = \sum_{i=1}^{n} [\text{null}_i \log(\pi_i) + (1 - \text{null}_i) \log(1 - \pi_i)] \tag{10}$$

In a next step, all π_is would be substituted by their respective probabilities from the right-hand side of Equation 4.

The goal is to find the value that maximises $\ell(\beta)$, i.e., the **maximum likelihood estimator** $\hat{\beta}$. The solution is found via iterative optimisation techniques (e.g., the Newton-Raphson method or gradient-based methods; see the last section of this handout). Sometimes the algorithm may fail to find an optimal solution, which R reports as a warning (e.g., `glm.fit: algorithm did not converge`). A common cause is (quasi-)complete separation, i.e., a predictor level that occurs with only one of the two outcomes. For further technical details, see Wood (2006: 63-66) or Agresti & Kateri (2022: 291-294).

Workflow in R

Research question and hypotheses

How do the intra- and extra-linguistic variables suggested in the literature affect subject pronoun realisation (Definite Null Instantiation) in British English, Singapore English and Hong Kong English?

Given the significance level $\alpha = 0.05$, the hypotheses are:

H_0 : None of the predictor coefficients deviate from 0.
H_1 : At least one predictor coefficient deviates from 0.

These can be restated mathematically as:

$$H_0 : \quad \beta_1 = \beta_2 = \cdots = \beta_p = 0$$
$$H_1 : \quad \text{At least one } \beta_i \neq 0 \text{ for } i \in \{1, 2, \ldots, p\}$$

Convert to factors and specify reference levels

The next step involves specifying **reference levels** for all categorical variables. This step is very important because it will directly impact the parameter estimation procedure and, consequently, influence our interpretation of the model output.

- The reference level of the response is usually chosen such that it corresponds to the **unmarked or most frequent case**. Since overt pronouns are much more common in the data, the reference level of the `Reference` variable will be set to `Reference = overt`. Since `glm()` always models the probability of the second (i.e., non-reference) level, the model will then predict the log odds (and hence the probability) of the **null subject** variant (i.e., the special case) given certain predictor configurations.
- The **predictor levels** need to be specified as well. Among other things, we are interested in how the Asian Englishes pattern relative to British English. Therefore, we will define British English as the baseline for comparison.

We will use the following specifications:

Variable	**Factor Levels**	**Preferred Reference level**
Register	S1A, S1B	S1A
Variety	GB, SING, HK	GB
Person	1, 2, 3, *it*, *there*	3
Referentiality	referential, non-referential	referential

```
# Store "Reference" as factor
data_pro$Reference <- as.factor(data_pro$Reference)

## Specify reference level (the 'unmarked' case)
data_pro$Reference <- relevel(data_pro$Reference, "overt")

## Print levels
levels(data_pro$Reference)
```

```
[1] "overt" "null"
```

Repeat the procedure for the remaining categorical variables.

```
# Store "Register" as factor
data_pro$Register <- as.factor(data_pro$Register)

## Specify reference level
data_pro$Register <- relevel(data_pro$Register, "S1A")

# Store "Variety" as factor
data_pro$Variety <- as.factor(data_pro$Variety)

## Specify reference level
data_pro$Variety <- relevel(data_pro$Variety, "GB")

# Store "Person" as factor
data_pro$Person <- as.factor(data_pro$Person)

## Specify reference level
data_pro$Person <- relevel(data_pro$Person, "3")

# Store "Referentiality" as factor
data_pro$Referentiality <- as.factor(data_pro$Referentiality)

## Specify reference level
data_pro$Referentiality <- relevel(data_pro$Referentiality, "referential")
```

Model fitting

```
# With (glm); available in base R
# Note the additional "family" argument!
Reference.glm <- glm(Reference ~ Register + Variety + Person +
 ↪  Referentiality, data = data_pro, family = "binomial")
```

```
# View model statistics
summary(Reference.glm)
```

```
Call:
glm(formula = Reference ~ Register + Variety + Person + Referentiality,
    family = "binomial", data = data_pro)

Coefficients: (1 not defined because of singularities)
                             Estimate Std. Error z value Pr(>|z|)
(Intercept)                   -3.4402     0.2221 -15.489  < 2e-16 ***
RegisterS1B                    0.1102     0.1603   0.688  0.49173
VarietyHK                      0.9856     0.2248   4.385 1.16e-05 ***
VarietySING                    0.9662     0.2272   4.252 2.12e-05 ***
Person1                       -0.9155     0.1800  -5.086 3.65e-07 ***
Person2                       -1.6676     0.2691  -6.198 5.72e-10 ***
Personit                       0.8130     0.2960   2.747  0.00602 **
Personthere                   -2.6367     1.0078  -2.616  0.00889 **
Referentialitynon-referential       NA         NA      NA       NA
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for binomial family taken to be 1)

    Null deviance: 1498.8  on 4837  degrees of freedom
Residual deviance: 1387.5  on 4830  degrees of freedom
AIC: 1403.5

Number of Fisher Scoring iterations: 7
```

```
# Get Pseudo-R^2 (requires the DescTools library)
DescTools::PseudoR2(Reference.glm, "Nagelkerke")
```

```
Nagelkerke
0.08538986
```

```
# Likelihood ratio test
lmtest::lrtest(Reference.glm)
```

```
Likelihood ratio test

Model 1: Reference ~ Register + Variety + Person + Referentiality
```

```
Model 2: Reference ~ 1
  #Df  LogLik Df  Chisq Pr(>Chisq)
1   8 -693.75
2   1 -749.42 -7 111.33  < 2.2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Further model diagnostics

Post-hoc evaluation of the logistic regression model is just as important as it is for linear regression. However, several assumptions can be relaxed. While independence of observations, a linear relationship between continuous predictors and the **log odds** of the response, as well as the absence of strongly correlated predictors remain essential, the stipulations on the distribution and constant variance of the residuals may be disregarded.

Multicollinearity can be inspected via the `vif()` function from the `car` package:

```
car::vif(Reference.glm)
```

```
Error in vif.default(Reference.glm): there are aliased coefficients in the model
```

The function refuses to compute variance inflation factors because the model contains **aliased coefficients**. This is an extreme form of multicollinearity: One predictor can be perfectly predicted from the others. The model summary above already hinted at this problem, as one coefficient was reported as `NA` ('not defined because of singularities'). Checking for linear dependencies via `alias()` reveals `Referentiality = non-referential` as the culprit:

```
alias(Reference.glm)
```

```
Model :
Reference ~ Register + Variety + Person + Referentiality

Complete :
                              (Intercept) RegisterS1B VarietyHK VarietySING
Referentialitynon-referential 0           0           0         0
                              Person1 Person2 Personit Personthere
Referentialitynon-referential 0       0       1        1
```

Since `Referentiality` is completely determined by `Person` (because the non-referential subjects coincide with the dummy pronouns *it* and *there*), refitting the model without `Referentiality` solves the problem:

```
Reference.glm2 <- glm(Reference ~ Register + Variety + Person, data =
↪  data_pro, family = "binomial")

summary(Reference.glm2)
```

```
Call:
glm(formula = Reference ~ Register + Variety + Person, family = "binomial",
    data = data_pro)

Coefficients:
            Estimate Std. Error z value Pr(>|z|)
(Intercept)  -3.4402     0.2221 -15.489  < 2e-16 ***
RegisterS1B   0.1102     0.1603   0.688  0.49173
VarietyHK     0.9856     0.2248   4.385 1.16e-05 ***
VarietySING   0.9662     0.2272   4.252 2.12e-05 ***
Person1      -0.9155     0.1800  -5.086 3.65e-07 ***
Person2      -1.6676     0.2691  -6.198 5.72e-10 ***
Personit      0.8130     0.2960   2.747  0.00602 **
Personthere  -2.6367     1.0078  -2.616  0.00889 **
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for binomial family taken to be 1)

    Null deviance: 1498.8  on 4837  degrees of freedom
Residual deviance: 1387.5  on 4830  degrees of freedom
AIC: 1403.5

Number of Fisher Scoring iterations: 7
```

The VIFs look considerably better now:

```
car::vif(Reference.glm2)
```

```
             GVIF Df GVIF^(1/(2*Df))
Register 1.036261  1        1.017969
Variety  1.020936  2        1.005193
Person   1.047852  4        1.005860
```

Removing that problematic variable does not affect the fit at all, since the information it contains is already captured by `Person`:

```
anova(Reference.glm, Reference.glm2)
```

```
Analysis of Deviance Table

Model 1: Reference ~ Register + Variety + Person + Referentiality
Model 2: Reference ~ Register + Variety + Person
  Resid. Df Resid. Dev Df Deviance Pr(>Chi)
1      4830     1387.5
2      4830     1387.5  0        0
```

Testing individual predictors

With the function `drop1()`, it is possible to remove each predictor from the full model one at a time and test whether its removal makes the model significantly worse (i.e., whether it significantly increases the deviance). Note that using such tests for (stepwise) variable selection is controversial, since the p-values of the final model no longer account for the selection process.

```
drop1(Reference.glm, test = "Chisq")
```

```
Single term deletions

Model:
Reference ~ Register + Variety + Person + Referentiality
               Df Deviance    AIC    LRT  Pr(>Chi)
<none>              1387.5 1403.5
Register        1   1388.0 1402.0  0.471    0.4926
Variety         2   1413.7 1425.7 26.179 2.067e-06 ***
Person          3   1471.1 1481.1 83.607 < 2.2e-16 ***
Referentiality  0   1387.5 1403.5  0.000
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

The output suggests that `Register` and `Referentiality` do not render the model substantially worse upon their removal.

Confidence intervals and odds ratios

```
# Tidy the model output
tidy_model <- tidy(Reference.glm2, conf.int = TRUE)
```

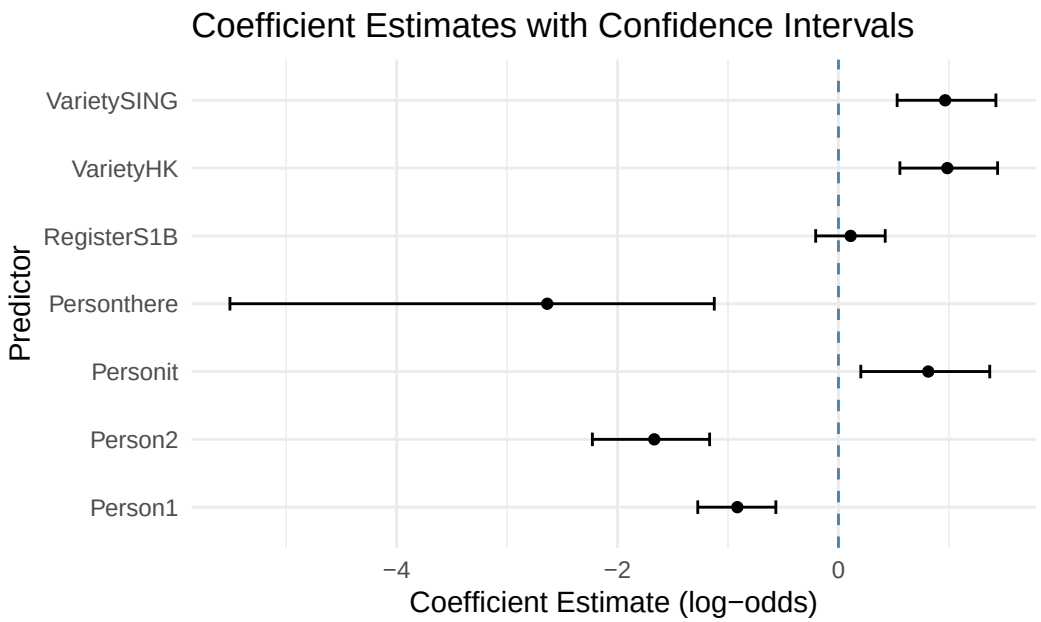
term	estimate	std.error	statistic	p.value	conf.low	conf.high	odds_ratio
RegisterS1B	0.1102450	0.1603417	0.6875627	0.4917282	-0.2061670	0.4233684	1.1165516
VarietyHK	0.9856195	0.2247932	4.3845615	0.0000116	0.5564413	1.4409284	2.6794714
VarietySING	0.9661514	0.2272470	4.2515473	0.0000212	0.5315430	1.4257107	2.6278115
Person1	-0.9155357	0.1799959	-5.0864250	0.0000004	-1.2738888	-0.5667152	0.4003021
Person2	-1.6676297	0.2690625	-6.1979274	0.0000000	-2.2280126	-1.1666544	0.1886938
Personit	0.8129708	0.2959570	2.7469221	0.0060157	0.2024969	1.3697902	2.2545961
Personthere	-2.6367446	1.0077634	-2.6164322	0.0088854	-5.5095493	-1.1242383	0.0715940

```
# Remove intercept, compute odds ratios and their CIs
# (alternatively, tidy(..., exponentiate = TRUE) returns odds ratios
 ↪  directly)
(tidy_model <- tidy_model %>%
  filter(term != "(Intercept)") %>%
  mutate(
    odds_ratio = exp(estimate),
    odds.conf.low = exp(conf.low),
    odds.conf.high = exp(conf.high)
  )
)
```

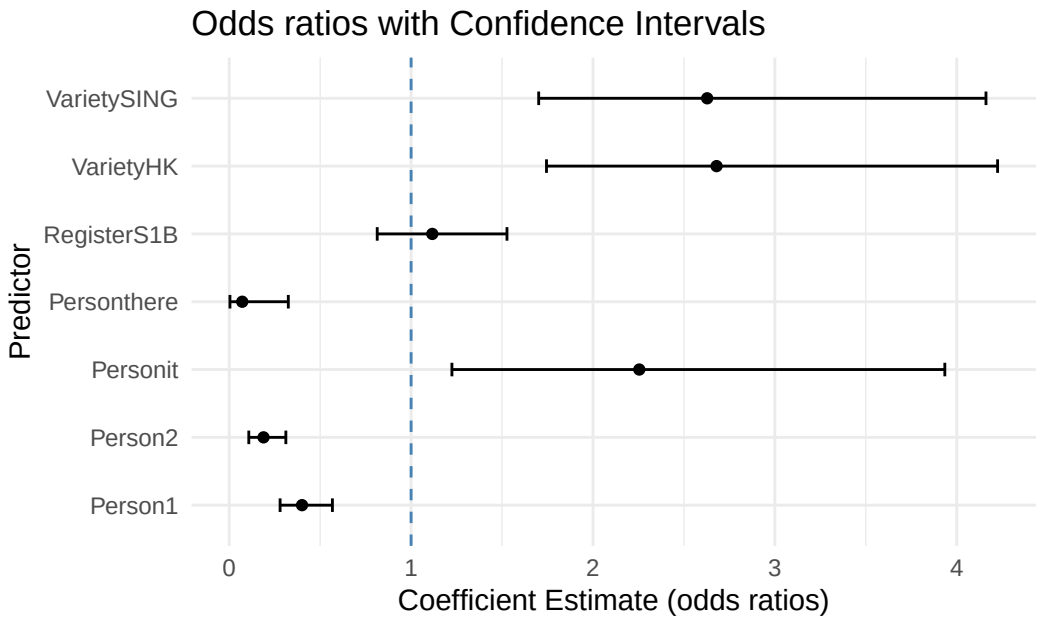
Visualisation

- Plot model coefficients:

```
# Create the coefficient plot
ggplot(tidy_model, aes(x = estimate, y = term)) +
  geom_point() +
  geom_errorbarh(aes(xmin = conf.low, xmax = conf.high), height = 0.2) +
  geom_vline(xintercept = 0, linetype = "dashed", color = "steelblue") +
  theme_minimal() +
  labs(
    x = "Coefficient Estimate (log-odds)",
    y = "Predictor",
    title = "Coefficient Estimates with Confidence Intervals",
    caption = "*Note that the CIs of significant predictors do not include
     ↪  0."
  )
```




```
# Plot odds ratios
ggplot(tidy_model, aes(x = exp(estimate), y = term)) +
  geom_point() +
  geom_errorbarh(aes(xmin = odds.conf.low, xmax = odds.conf.high), height =
   ↪  0.2) +
  geom_vline(xintercept = 1, linetype = "dashed", color = "steelblue") +
  theme_minimal() +
  labs(
    x = "Coefficient Estimate (odds ratios)",
    y = "Predictor",
    title = "Odds ratios with Confidence Intervals",
    caption = "*Note that the CIs of significant predictors do not include
     ↪  1."
  )
```



Interpret the model

The full logistic regression model is significantly better than a null model with no predictors (likelihood ratio test: $\chi^2(7) = 111.33$, $p < 0.001$), although its explanatory power is modest (Nagelkerke's $R^2 = 0.09$). However, perfect collinearity has been detected and traced back to `Referentiality`; removing this feature resolved the issue and did not affect model fit.

The model coefficients of the reduced model indicate that null subjects are significantly more likely in Singapore English compared to British English (Estimate = 0.97, 95% CI [0.53, 1.43], $p < 0.001$). This effect is moderate with an OR of 2.63 (95% CI [1.70, 4.16]), suggesting that the odds of subject omission are approximately 2.6 times higher in Singapore English than in British English, holding the other predictors constant.

...

Looking under the hood

Some hidden metrics

Beside the coefficients, the model object `Reference.glm2` stores a variety of additional metrics, some of which appear to be rather cryptic:

```
# A tibble: 1 x 8
  null.deviance df.null logLik   AIC   BIC deviance df.residual  nobs
          <dbl>   <int>  <dbl> <dbl> <dbl>    <dbl>       <int> <int>
1         1499.    4837  -694. 1404. 1455.    1388.        4830  4838
```

- `logLik` is the log-likelihood of the model at the maximum likelihood estimate $\ell(\hat{\beta})$ (cf. Equation 9). Closer to 0 (less negative) means a better fit; however, this measure isn't typically reported.

```
logLik(Reference.glm2)
```

```
'log Lik.' -693.7532 (df=8)
```

- `deviance` (residual deviance) is $-2\ell(\hat{\beta})$ for the fitted model. It's the logistic-regression analogue of the residual sum of squares in linear regression. Once again, smaller values indicate better fit.

```
Reference.glm2$deviance
```

```
[1] 1387.506
```

```
-2*logLik(Reference.glm2)
```

```
'log Lik.' 1387.506 (df=8)
```

- `null.deviance` is the deviance of an intercept-only model with no predictors.

```
Reference.glm2$null.deviance
```

```
[1] 1498.836
```

These two deviance measures can be used to obtain Pseudo-R^2 measures, which assess the overall fit of the model.

```
n <- nrow(Reference.glm2$model)
D_model <- Reference.glm2$deviance
D_null <- Reference.glm2$null.deviance

# Get Cox & Snell's R^2; it's not used widely because of its awkward scale
R2_CS  <- 1 - exp(-(D_null - D_model) / n)

# Equivalent to
PseudoR2(Reference.glm2, "CoxSnell")
```

```
 CoxSnell
0.0227487
```

```
# Rescale to [0,1]
R2_max <- 1 - exp(-D_null / n) # get maximum possible value Cox & Snell's
 ↪  R^2 can take on
R2_N   <- R2_CS / R2_max # normalise

# Equivalent to
PseudoR2(Reference.glm2, "Nagelkerke")
```

```
Nagelkerke
0.08538986
```

- `AIC/BIC`: Akaike Information Criterion/Bayesian Information Criterion. AIC ($-2\ell(\hat{\beta}) + 2 \times$ (number of parameters)) penalises the model for including too many predictors, especially if they are uninformative; this penalty is even stronger in the BIC. Typically, models with lower AIC/BIC are preferred.

```
AIC(Reference.glm2)
```

```
[1] 1403.506
```

```
# Number of parameters = number of estimated coefficients (including
 ↪  intercept)
n_params <- length(coef(Reference.glm2))

# Calculate AIC manually
-2*logLik(Reference.glm2) + 2*n_params
```

```
'log Lik.' 1403.506 (df=8)
```

- `df.null` and `df.residual`: Degrees of freedom for the null model and the fitted model, respectively. They correspond to $n - 1$ and $n -$ number of parameters.

```
# Example: residual df
length(Reference.glm2$y) - n_params
```

```
[1] 4830
```

- `nobs`: Number of observations n.

```
length(Reference.glm2$y)
```

```
[1] 4838
```

How do you actually estimate β in logistic regression?

Recall the log-likelihood function:

$$\ell(\beta) = \sum_{i=1}^{n} [y_i \log(\pi_i) + (1 - y_i) \log(1 - \pi_i)]$$

We need to find the parameter value of β for which the likelihood is as high as possible. This is achieved by setting all derivatives to zero:

$$\frac{\partial \ell(\beta)}{\partial \beta_j} = \sum_{i=1}^{n} x_{ij}(y_i - \pi_i) = 0 \quad \text{for } j = 0, 1, \ldots, p$$

These are known as the score equations. As there is no closed-form solution for this problem, we will need to use an algorithm that will provide us with an approximate answer. A popular approach is the Newton-Raphson method, which requires us to compute a matrix of second partial derivatives known as the **Hessian**:

$$\frac{\partial^2 \ell(\beta)}{\partial \beta \partial \beta^\top} = -\mathbf{X}^\top \mathbf{W} \mathbf{X} \quad \text{where } \mathbf{W} = \text{diag}(\pi_i(1 - \pi_i))$$

The algorithm continuously updates its current solution using the following rule until it finds a stable solution:

$$\beta^{\text{new}} = \beta^{\text{old}} - \left(\frac{\partial^2 \ell(\beta)}{\partial \beta \partial \beta^\top} \right)^{-1} \frac{\partial \ell(\beta)}{\partial \beta},$$

usually starting at 0. For illustration, you can find a strongly simplified and computationally inefficient R implementation of this algorithm below; nevertheless, the parameter estimates are virtually identical to the ones produced by `glm()`.

```
X <- model.matrix(Reference ~ Register + Variety + Person, data = data_pro)
y <- as.numeric(data_pro$Reference == "null")

# Initialise beta at 0
beta <- rep(0, ncol(X))

# Newton-Raphson (for logistic regression equivalent to "iteratively
 ↪  reweighted least squares")
for (iter in 1:20) {

  p_hat <- plogis(X %*% beta)                    # pi_i = predicted
 ↪  probabilities
```

	term	manual_coef	glm_coef
(Intercept)	(Intercept)	-3.440203	-3.440203
RegisterS1B	RegisterS1B	0.110245	0.110245
VarietyHK	VarietyHK	0.985620	0.985620
VarietySING	VarietySING	0.966151	0.966151
Person1	Person1	-0.915536	-0.915536
Person2	Person2	-1.667630	-1.667630
Personit	Personit	0.812971	0.812971
Personthere	Personthere	-2.636748	-2.636745

```
  score <- t(X) %*% (y - p_hat)                # score equations X^T(y -
↪  pi)

  W <- diag(as.vector(p_hat * (1 - p_hat)))    # weights W = diag(pi_i(1 -
↪  pi_i))

  hessian <- -t(X) %*% W %*% X                  # Hessian -X^T W X

  beta <- beta - solve(hessian) %*% score       # Newton step
}
```

References

Agresti, Alan, and Maria Kateri. 2022. *Foundations of Statistics for Data Scientists: With r and Python*. Boca Raton: CRC Press.

Buskin, Vladimir. 2025. "Definite Null Instantiation in English(es): A Usage-based Construction Grammar approach." *Constructions and Frames*. https://doi.org/10.1075/cf.24007.bus.

Hosmer, David W., and Stanley Lemeshow. 2008. *Applied Logistic Regression*. 2nd ed. New York: Wiley.

James, Gareth, Daniela Witten, Trevor Hastie, and Robert Tibshirani. 2021. *An Introduction to Statistical Learning: With Applications in r*. New York: Springer. https://doi.org/10.1007/978-1-0716-1418-1.

Levshina, Natalia. 2015. *How to Do Linguistics with r: Data Exploration and Statistical Analysis*. Amsterdam; Philadelphia: John Benjamins Publishing Company.

Winter, Bodo. 2020. *Statistics for Linguists: An Introduction Using r*. New York; London: Routledge.

Wood, Simon N. 2006. *Generalized Additive Models: An Introduction with R*. Boca Raton: Chapman & Hall/CRC.